

Assignment 3

Prompt

Generate C++ code using objects to prioritize emails for a CEO. Implement a MaxHeap from scratch using a list-based implementation; do not use a pre-existing heap module. Each email is read from a line formatted as `EMAIL <sender category>,<subject line>,<date>`, with the date in `MM-DD-YYYY` format. The subject may contain spaces but not commas.

Prioritize sender categories in this order: `Boss`, `Subordinate`, `Peer`, `ImportantPerson`, and `OtherPerson`. For emails from the same category, the newest date comes first. `NEXT` displays the highest-priority email without removing it, `READ` removes the highest-priority email, and `COUNT` displays the number of unread emails. Commands may occur in varying orders, including when the queue is empty. The sample contains six emails and should show the newest Boss email first, then the older Boss email after a `READ`, and end with four unread emails.

Gemini Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>

// Class to represent individual Email data and comparison logic
class Email {
private:
    std::string senderCategory;
    std::string subject;
    std::string date; // Format: MM-DD-YYYY

    // Helper to extract numerical priority from sender category
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        if (senderCategory == "OtherPerson") return 1;
        return 0;
    }

    // Helper to parse MM-DD-YYYY date into YYYYMMDD integer for comparison
    int getFormattedDate() const {
        if (date.length() != 10) return 0;
        int month = std::stoi(date.substr(0, 2));
        int day = std::stoi(date.substr(3, 2));
        int year = std::stoi(date.substr(6, 4));
        return year * 10000 + month * 100 + day;
    }
}
```

```

public:
    Email() : senderCategory(""), subject(""), date("") {}

    Email(const std::string& cat, const std::string& sub, const std::string& d)
        : senderCategory(cat), subject(sub), date(d) {}

    std::string getSenderCategory() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }

    // Comparison operator overloading for MaxHeap ordering:
    // Higher category priority comes first.
    // If categories match, newer date (larger YYYYMMDD) comes first.
    bool operator>(const Email& other) const {
        int p1 = getCategoryPriority();
        int p2 = other.getCategoryPriority();

        if (p1 != p2) {
            return p1 > p2;
        }
        return getFormattedDate() > other.getFormattedDate();
    }

    bool operator<(const Email& other) const {
        return other > *this;
    }
};

// Custom MaxHeap implementation built from scratch without external libraries
class MaxHeap {
private:
    Email* heapArray;
    int capacity;
    int heapSize;

    void resize() {
        capacity *= 2;
        Email* newArray = new Email[capacity];
        for (int i = 0; i < heapSize; ++i) {
            newArray[i] = heapArray[i];
        }
        delete[] heapArray;
        heapArray = newArray;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heapArray[index] > heapArray[parentIndex]) {
                Email temp = heapArray[index];
                heapArray[index] = heapArray[parentIndex];
                heapArray[parentIndex] = temp;
                index = parentIndex;
            } else {

```

```

        break;
    }
}

void heapifyDown(int index) {
    int largest = index;
    int leftChild = 2 * index + 1;
    int rightChild = 2 * index + 2;

    if (leftChild < heapSize && heapArray[leftChild] > heapArray[largest]) {
        largest = leftChild;
    }
    if (rightChild < heapSize && heapArray[rightChild] > heapArray[largest]) {
        largest = rightChild;
    }

    if (largest != index) {
        Email temp = heapArray[index];
        heapArray[index] = heapArray[largest];
        heapArray[largest] = temp;
        heapifyDown(largest);
    }
}

public:
    MaxHeap(int initialCapacity = 10) : capacity(initialCapacity), heapSize(0) {
        heapArray = new Email[capacity];
    }

    ~MaxHeap() {
        delete[] heapArray;
    }

    void insert(const Email& email) {
        if (heapSize == capacity) {
            resize();
        }
        heapArray[heapSize] = email;
        heapifyUp(heapSize);
        heapSize++;
    }

    Email peek() const {
        if (heapSize == 0) {
            return Email();
        }
        return heapArray[0];
    }

    void extractMax() {
        if (heapSize == 0) return;
        heapArray[0] = heapArray[heapSize - 1];
        heapSize--;
    }

```

```
        if (heapSize > 0) {
            heapifyDown(0);
        }
    }

    int size() const {
        return heapSize;
    }

    bool isEmpty() const {
        return heapSize == 0;
    }
};

// Priority Queue wrapper class utilizing the custom MaxHeap
class EmailPriorityQueue {
private:
    MaxHeap maxHeap;

public:
    void enqueue(const Email& email) {
        maxHeap.insert(email);
    }

    Email peekNext() const {
        return maxHeap.peak();
    }

    void dequeue() {
        maxHeap.extractMax();
    }

    int count() const {
        return maxHeap.size();
    }

    bool isEmpty() const {
        return maxHeap.isEmpty();
    }
};

// Command Controller class to manage commands and parse input test files
class EmailManager {
private:
    EmailPriorityQueue emailQueue;

    void trim(std::string& str) {
        size_t first = str.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) {
            str = "";
            return;
        }
        size_t last = str.find_last_not_of(" \t\r\n");
        str = str.substr(first, (last - first + 1));
    }
};
```

```
}

public:
    void processCommand(const std::string& line) {
        if (line.empty()) return;

        if (line.rfind("EMAIL", 0) == 0) {
            std::string data = line.substr(5);
            trim(data);

            std::stringstream ss(data);
            std::string category, subject, date;

            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {

                trim(category);
                trim(subject);
                trim(date);

                Email email(category, subject, date);
                emailQueue.enqueue(email);
            }
        } else if (line == "COUNT") {
            std::cout << "There are " << emailQueue.count() << " emails to read."
<< std::endl;
        } else if (line == "NEXT") {
            if (!emailQueue.isEmpty()) {
                Email nextEmail = emailQueue.peekNext();
                std::cout << "Next email:" << std::endl;
                std::cout << "Sender: " << nextEmail.getSenderCategory() <<
std::endl;

                std::cout << "Subject: " << nextEmail.getSubject() << std::endl;
                std::cout << "Date: " << nextEmail.getDate() << std::endl;
            } else {
                std::cout << "No emails in queue." << std::endl;
            }
        } else if (line == "READ") {
            if (!emailQueue.isEmpty()) {
                emailQueue.dequeue();
            }
        }
    }

    void processFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error opening file: " << filename << std::endl;
            return;
        }

        std::string line;
        while (std::getline(file, line)) {
```

```

        trim(line);
        if (!line.empty()) {
            processCommand(line);
        }
    }
    file.close();
}
};

int main(int argc, char* argv[]) {
    EmailManager manager;

    // Use command line argument if provided, otherwise default to "test.txt"
    std::string filename = (argc > 1) ? argv[1] : "test.txt";
    manager.processFile(filename);

    return 0;
}

```

The implementation is in [Gemini.cpp](#). It uses four classes: `Email` stores the fields and defines ordering, `MaxHeap` implements the heap in a dynamically allocated array, `EmailPriorityQueue` wraps the heap operations, and `EmailManager` parses commands and manages the input file. The array capacity doubles when full. `Email::operator<` compares category priority and then a numeric `YYYYMMDD` date value. The program reads a filename argument or defaults to `test.txt`.

ChatGPT Code

```

#include <iostream>
#include <string>
#include <sstream>
#include <vector>

using namespace std;

// -----
// Email class
// Represents one email in the CEO's inbox.
// -----
class Email
{
private:
    string sender;
    string subject;
    string date;

public:
    Email()
    {
        sender = "";
        subject = "";
        date = "";
    }
}

```

```
}

Email(string s, string sub, string d)
{
    sender = s;
    subject = sub;
    date = d;
}

string getSender()
{
    return sender;
}

string getSubject()
{
    return subject;
}

string getDate()
{
    return date;
}

// Returns the priority of the sender category.
// Higher number = higher priority.
int getSenderPriority()
{
    if (sender == "Boss")
        return 5;
    else if (sender == "Subordinate")
        return 4;
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1;    // OtherPerson
}

// Converts MM-DD-YYYY into YYYYMMDD.
// This makes dates easy to compare numerically.
int getDateValue()
{
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

// Determines whether this email has a higher priority
// than another email.
bool higherPriorityThan(Email other)
```

```
{
    // First compare sender category.
    if (getSenderPriority() != other.getSenderPriority())
    {
        return getSenderPriority() > other.getSenderPriority();
    }

    // If sender category is the same, newer date wins.
    if (getDateValue() != other.getDateValue())
    {
        return getDateValue() > other.getDateValue();
    }

    // If everything is equal, neither has priority.
    return false;
}
};

// -----
// MaxHeap class
// List-based implementation of a MaxHeap.
// The vector is used as the list that stores the heap.
// -----
class MaxHeap
{
private:
    vector<Email> heap;

public:

    // Returns the index of the parent.
    int parent(int index)
    {
        return (index - 1) / 2;
    }

    // Returns the index of the left child.
    int leftChild(int index)
    {
        return (2 * index) + 1;
    }

    // Returns the index of the right child.
    int rightChild(int index)
    {
        return (2 * index) + 2;
    }

    // Moves an item upward until the MaxHeap property is restored.
    void heapifyUp(int index)
    {
        while (index > 0)
        {
```



```
        int parentIndex = parent(index);

        if (heap[index].higherPriorityThan(heap[parentIndex]))
        {
            Email temporary = heap[index];
            heap[index] = heap[parentIndex];
            heap[parentIndex] = temporary;

            index = parentIndex;
        }
        else
        {
            break;
        }
    }
}

// Moves an item downward until the MaxHeap property is restored.
void heapifyDown(int index)
{
    int size = heap.size();

    while (true)
    {
        int left = leftChild(index);
        int right = rightChild(index);
        int largest = index;

        if (left < size &&
            heap[left].higherPriorityThan(heap[largest]))
        {
            largest = left;
        }

        if (right < size &&
            heap[right].higherPriorityThan(heap[largest]))
        {
            largest = right;
        }

        if (largest != index)
        {
            Email temporary = heap[index];
            heap[index] = heap[largest];
            heap[largest] = temporary;

            index = largest;
        }
        else
        {
            break;
        }
    }
}
```

```
// Adds an email to the MaxHeap.
void insert(Email email)
{
    heap.push_back(email);

    int index = heap.size() - 1;
    heapifyUp(index);
}

// Returns the highest-priority email without deleting it.
Email peek()
{
    return heap[0];
}

// Removes the highest-priority email.
Email removeMax()
{
    Email highestPriority = heap[0];

    heap[0] = heap[heap.size() - 1];
    heap.pop_back();

    if (heap.size() > 0)
    {
        heapifyDown(0);
    }

    return highestPriority;
}

// Returns the number of unread emails.
int size()
{
    return heap.size();
}

// Returns true if there are no emails.
bool isEmpty()
{
    return heap.empty();
}
};

// -----
// CEOInbox class
// Handles the CEO's commands and uses the MaxHeap.
// -----
class CEOInbox
{
private:
    MaxHeap emailQueue;
```

```
public:

    // Adds an email to the CEO's inbox.
    void addEmail(string sender, string subject, string date)
    {
        Email newEmail(sender, subject, date);
        emailQueue.insert(newEmail);
    }

    // Displays the highest-priority email without removing it.
    void nextEmail()
    {
        if (emailQueue.isEmpty())
        {
            return;
        }

        Email email = emailQueue.peek();

        cout << "Next email:" << endl;
        cout << "Sender: " << email.getSender() << endl;
        cout << "Subject: " << email.getSubject() << endl;
        cout << "Date: " << email.getDate() << endl;
    }

    // Removes the highest-priority email.
    void readEmail()
    {
        if (emailQueue.isEmpty())
        {
            return;
        }

        emailQueue.removeMax();
    }

    // Displays the number of unread emails.
    void countEmails()
    {
        cout << "There are " << emailQueue.size()
              << " emails to read." << endl;
    }

    // Processes one command from the input.
    void processCommand(string command)
    {
        if (command == "NEXT")
        {
            nextEmail();
        }
        else if (command == "READ")
        {
            readEmail();
        }
    }
}
```

```
    }
    else if (command == "COUNT")
    {
        countEmails();
    }
}

};

// -----
// Main program
// -----
int main()
{
    CEOInbox inbox;

    string line;

    // Read the entire test file until EOF.
    while (getline(cin, line))
    {
        // Ignore empty lines.
        if (line.empty())
        {
            continue;
        }

        // -----
        // EMAIL command
        // -----
        if (line.substr(0, 6) == "EMAIL ")
        {
            // Remove "EMAIL " from the beginning.
            string emailData = line.substr(6);

            // Find the first comma.
            int firstComma = emailData.find(',');

            // Find the second comma.
            int secondComma = emailData.find(',', firstComma + 1);

            // Extract sender.
            string sender =
                emailData.substr(0, firstComma);

            // Extract subject.
            string subject =
                emailData.substr(
                    firstComma + 1,
                    secondComma - firstComma - 1
                );

            // Extract date.
            string date =
```

```

        emailData.substr(secondComma + 1);

        // Add email to the MaxHeap.
        inbox.addEmail(sender, subject, date);
    }

    // -----
    // Other commands
    // -----
    else
    {
        inbox.processCommand(line);
    }
}

return 0;
}

```

The implementation is in [GPT.cpp](#). It uses three classes: `Email` stores email data and compares priorities, `MaxHeap` implements the heap using `std::vector<Email>`, and `CEOInbox` handles queue operations and commands. Its `Email::higherPriorityThan` compares sender category and then date. The program reads commands from standard input, allowing a test file to be provided with input redirection.

Analysis

Gemini

Correctness:

The category ordering matches the prompt, and the date conversion makes later dates compare as higher priority within the same category. `NEXT` peeks at the heap without removing the email. `READ` removes the heap maximum, whether or not `NEXT` was called first, and `COUNT` reports the current heap size. Repeated `NEXT` commands therefore show the same email; repeated `READ` commands remove successive highest-priority emails. On an empty queue, `NEXT` prints `No emails in queue.` and `READ` does nothing. The parser trims whitespace from category, subject, and date fields.

The date conversion assumes the date has numeric month, day, and year components. Invalid date text can make `std::stoi` throw, and the code does not reject impossible calendar dates. The assignment guarantees the specified date format, so this is outside its stated input assumptions.

Execution Time:

Insertion and removal each take $O(\log n)$ in the worst case for a heap containing n emails. `NEXT` and `COUNT` take $O(1)$. Inserting n input emails takes $O(n \log n)$ in the worst case.

Space Complexity:

The heap stores all unread emails and therefore uses $O(n)$ space. The backing array doubles its capacity when it fills.

Maintainability:

The separate classes give email data, heap behavior, queue operations, and command processing distinct roles. The comparison operator keeps the heap ordering concise. The raw array requires manual memory management. Because `MaxHeap` has a destructor but does not define copy construction or copy assignment, an accidental copy would share the pointer and could cause double deletion. The current `EmailPriorityQueue` uses its heap as a member and does not intentionally copy it.

ChatGPT

Correctness:

The category ordering and newest-date-first comparison match the prompt. `nextEmail` peeks without removing, `readEmail` removes the maximum, and `countEmails` reports the current size. Thus repeated `NEXT` and repeated `READ` work as described. `CEOInbox` checks for an empty queue before calling `peek` or `removeMax`, so empty `NEXT` and `READ` produce no output and do not index an empty vector. The prompt requires handling these cases but does not prescribe a message.

The parser extracts the sender, subject, and date using comma positions, but does not check that both commas were found before using the positions. It also does not trim field whitespace. A sender with leading or trailing spaces will not match a category and will be treated as the lowest priority by `getSenderPriority`. Like Gemini, the date conversion assumes parseable numeric date fields and does not validate calendar dates.

Execution Time:

Insertion and removal take $O(\log n)$ in the worst case. Peeking at the next email and getting the count take $O(1)$. Loading n emails takes $O(n \log n)$ in the worst case.

Space Complexity:

The vector-backed heap stores n unread emails and uses $O(n)$ space. `std::vector` manages its storage as the heap grows.

Maintainability:

The class structure is straightforward, and using `std::vector` avoids manual allocation and deallocation. The priority comparison is explicit and readable. The heap's `peek` and `removeMax` methods themselves assume the heap is not empty; the inbox currently protects these calls, but the public methods would be safer if they handled emptiness directly. The source also uses `using namespace std`, which is convenient in a small assignment but can cause naming conflicts in larger programs.

Overall Comparison

Both programs use classes and implement their own `MaxHeap` rather than importing a heap module. Their priority ordering, queue operation costs, and asymptotic space usage are the same. Both should produce the sample's required ordering and final count.

The main structural difference is storage: Gemini manages a resizable raw array, while GPT uses `std::vector`. The vector version needs less manual memory-management code. Gemini also has an extra `EmailPriorityQueue` wrapper class and accepts a filename argument (defaulting to `test.txt`); GPT reads from standard input. These are different ways of supplying the test file, not differences in priority behavior.

Input handling is the main behavioral distinction. Gemini trims whitespace around the three parsed fields and prints a message for an empty **NEXT**. GPT does not trim fields and prints nothing for empty **NEXT**. Both ignore an empty **READ** safely through checks in their command-handling classes. GPT's parser does not validate comma positions before extracting substrings, while Gemini checks that each comma-delimited field was read successfully. Neither program validates malformed or impossible dates, although the assignment says to assume the specified date format.

Overall, GPT has simpler heap storage, while Gemini is more tolerant of whitespace in email fields and checks that the parsed fields exist. The exact empty-queue output is not specified, so Gemini's message and GPT's silence are both reasonable interpretations. For a stronger solution, GPT should validate the comma positions and trim fields, and Gemini should define safe copy behavior for its raw-array heap; both could validate dates if inputs outside the prompt's assumptions needed to be supported.

My Selection

For the assignment, I selected GPT's program because I preferred the way the emails were stored compared to the Copilot version. To improve some of the issues in this code, I will add checks for the comma-delimited fields, trim whitespace from email data, and handle **NEXT** explicitly when the inbox is empty.